

Student

Chapter 1 taught me that ML learns from data. But **what is data in ML**, how do I get it, and how do I prepare it for training?

ChatGPT • AI Assistant

Excellent - this is where beginners meet reality. Chapter 1 said data work is **~70% of every ML project**; this chapter is where you learn the tools and vocabulary of that 70%. Ten focused subsections:

1. **ML terminology** - datasets, features, labels, samples, dimensions.
2. **Training, validation, and test sets** - the three splits every ML project needs.
3. **Numerical vs categorical data** - continuous, discrete, nominal, ordinal.
4. **Special data types** - time series, text, image, audio, video, graph.
5. **Data collection from structured sources** - CSV, Excel, SQL databases, warehouses.
6. **Data collection from live sources** - APIs, sensors, IoT, web scraping.
7. **Data cleaning** - missing values, duplicates, outliers.
8. **Feature scaling** - standardization and normalization.
9. **Encoding and feature engineering** - label encoding, one-hot, feature selection.
10. **Chapter review & summary.**

CONCEPT CARD

The vocabulary of ML data in one line each:

- **Dataset** - the full collection of examples you'll learn from.
- **Sample / observation / instance / row** - one example (a house, an email, a patient).
- **Feature / attribute / column / X** - one input variable per sample.
- **Label / target / y** - the output you want to predict.
- **Dimensions** - number of features per sample.

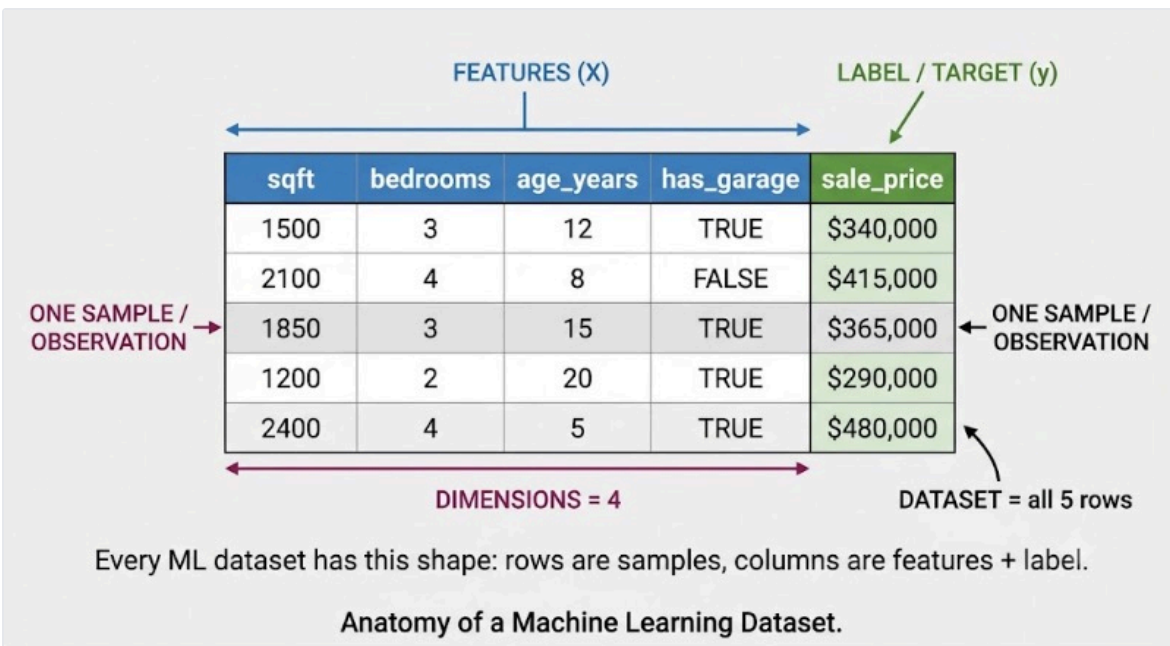
SUMMARY CARD

By the end of this chapter you'll be able to:

- Load data from CSV, Excel, SQL, and APIs into a pandas DataFrame.

- Split any dataset into training, validation, and test sets correctly.
- Identify the right preprocessing steps for numerical, categorical, and special data types.
- Clean a real dataset - handle missing values, duplicates, and outliers.
- Scale features and encode categorical variables using scikit-learn.

[AI Prompt] Warm-up (paste into ChatGPT or Claude): *"Explain the vocabulary of machine learning data - dataset, feature, label, sample, dimension - using one concrete real-world example. Then explain why the train/validation/test split matters."*



Anatomy of a Machine Learning Dataset

ML Data Vocabulary — Same Concept, Many Names

Concept	Aliases	Symbol	Example
Full dataset	Corpus, data table	D	houses.csv (10,000 rows)
One sample	Observation, instance, row, record	x	One house
Feature	Attribute, column, predictor, variable	X	sqft, bedrooms, age
Label	Target, output, ground truth, response	y	sale_price

Concept	Aliases	Symbol	Example
Dimension	Feature count, number of columns	d	4 features = 4 dimensions

python – A dataset as pandas sees it – the practical view

```
import pandas as pd

# A tiny house-price dataset in tabular form
data = {
    'sqft':      [1500, 2400, 900, 1800, 2100],
    'bedrooms':  [3, 4, 2, 3, 4],
    'age_years': [12, 5, 30, 8, 15],
    'has_garage': [True, True, False, True, True],
    'sale_price': [340000, 520000, 180000, 400000, 460000] # ←
}
label
df = pd.DataFrame(data)

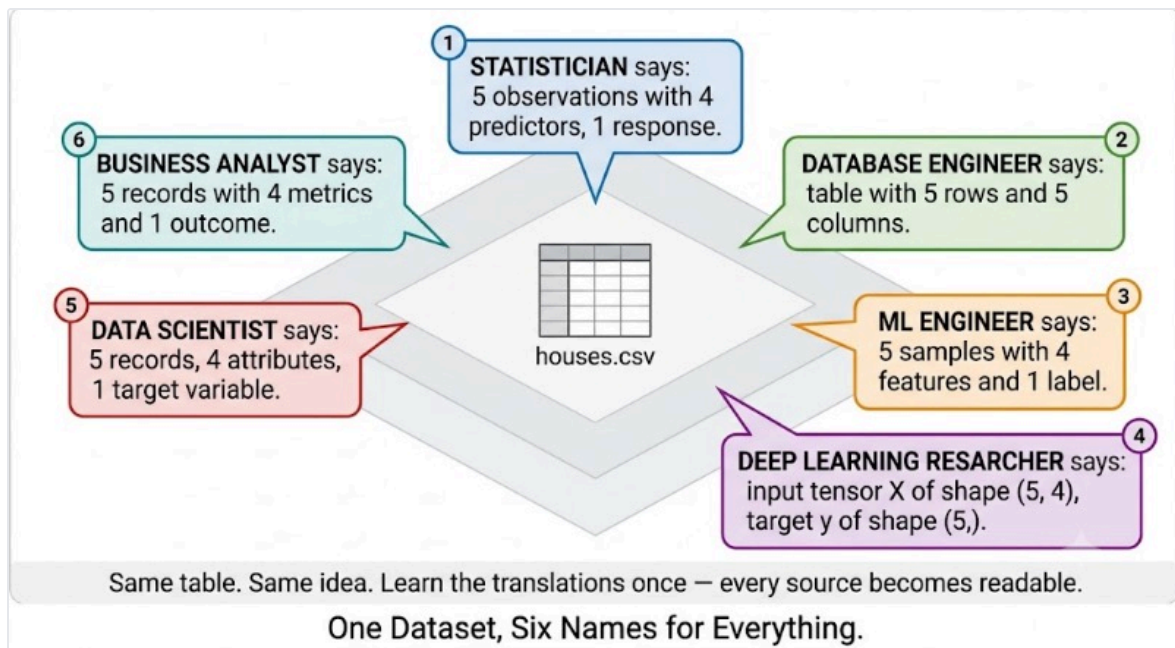
# Shape: (samples, features + label)
print(df.shape) # (5, 5)

# Features (X) and label (y) – the classic ML split
X = df[['sqft', 'bedrooms', 'age_years', 'has_garage']]
y = df['sale_price']

print(f'Dataset: {df.shape[0]} samples, {X.shape[1]} features')
print(f'Label range: ${y.min():,} to ${y.max():,}')
```

ML TERMINOLOGY — DATASETS, FEATURES, LABELS, SAMPLES

ChatGPT • AI Assistant



One Dataset, Six Names for Everything

Student

Give me precise definitions for every ML data term I'll hear.

ChatGPT • AI Assistant

ML has a rich vocabulary, but most terms describe the **same thing at different zoom levels**. Master these once and every paper, tutorial, and interview becomes easier.

CONCEPT CARD

The universal vocabulary:

1. **Dataset (D)** - the full collection of examples used to train or evaluate a model.
Think spreadsheet, CSV, or database table.
2. **Sample / Observation / Instance / Row / Record** - one example. All five words mean the same thing.
3. **Feature / Attribute / Column / Predictor / Variable / Input (X)** - one measurable property per sample. Multiple aliases; identical meaning.
4. **Label / Target / Output / Ground Truth / Response (y)** - the value you're trying to predict.
5. **Dimension / Feature Count / d** - number of features per sample.

6. Target Variable - same as label; the answer you want.

Student

Why so many synonyms?

ChatGPT • AI Assistant

EXAMPLE CARD

Different communities use different terms:

1. **Statisticians** - "predictors," "response," "observations."
2. **Database engineers** - "columns," "rows," "records."
3. **ML engineers** - "features," "labels," "samples."
4. **Deep learning researchers** - "inputs," "targets," "instances."

All describe the same underlying object - a table with rows and columns. Learn to translate between them and read anything in the field.

🔗 ANALOGY

An ML dataset is like a **class attendance sheet with grades**:

1. Each row = one student (**sample**).
2. Each column = one property (**feature**) - name, roll number, homework score.
3. Final-grade column = what you want to predict (**label**).
4. Number of property columns = **dimensions**.
5. Whole sheet = **dataset**.

⚠️ WARNING

Beginners often confuse *feature* with *label*. Features are the INPUTS (what the model sees); the label is the OUTPUT (what it must predict). Get this wrong once in code and your model "predicts" the answer trivially - a classic bug called **label leakage**.

🚫 COMMON PITFALL / MISTAKE

Incorrect Way: Including the label as one of the features when training. Model achieves 100% accuracy in training and 0% in production.

Correct Way: Always separate **x** (features) from **y** (label) before training. Never let **y** sneak into **x**.

EXAM TIP CARD

Interview classic: *"What's the difference between a feature and a label?"* Winning answer: *"A feature is an input variable the model uses to make predictions; a label is the output variable the model is trained to predict. Together they form a supervised-learning dataset."*

🔥 EXERCISE

Take any dataset you have (or download the Iris dataset). Identify: (a) number of samples, (b) number of features, (c) which column is the label, (d) how many dimensions it has.

[AI Prompt] Paste this into any chatbot: *"Give me 5 real-world datasets (with URLs on Kaggle or UCI). For each, identify the samples, features, label, and dimensions. Format as a table."*

🔗 PRACTICE QUIZ

In a dataset with 10,000 emails classified as spam or not-spam, using 20 word-count features per email, what are the samples, features, label, and dimensions?

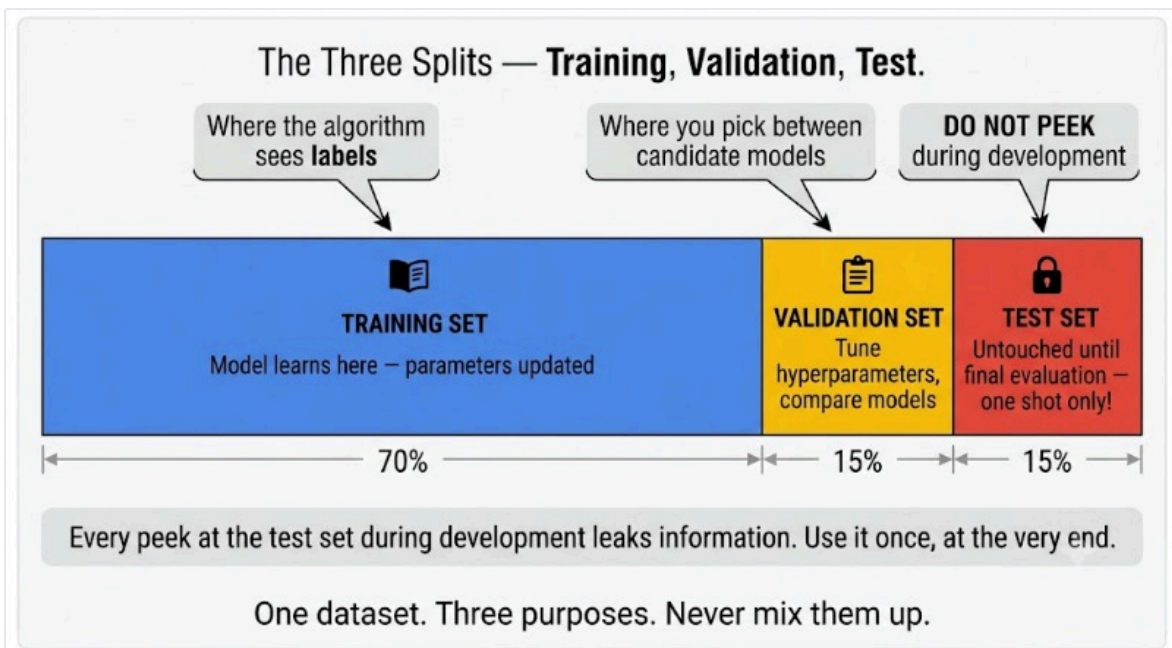
Answer: Samples = 10,000 emails; features = 20 word counts; label = spam/not-spam column; dimensions = 20.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

TRAINING, VALIDATION, AND TEST SETS

ChatGPT • AI Assistant



The Three Splits — Training, Validation, Test

python — Splitting a dataset into training, validation, and test sets

```

from sklearn.model_selection import train_test_split
import pandas as pd

# Load your dataset
df = pd.read_csv('houses.csv')
X = df.drop('sale_price', axis=1)          # features
y = df['sale_price']                      # label

# — STEP 1: Split off the test set first (15%) —
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y,
    test_size=0.15,
    random_state=42                        # reproducibility
)

# — STEP 2: Split the remaining 85% into train / val —
# We want 70% train and 15% val out of the ORIGINAL,
# which means val = 15/85 = ~0.176 of the temp split.
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=0.176,
    random_state=42
)

print(f'Training set:  {len(X_train):>5} samples

```

```
({len(X_train)/len(X):.0%})')
print(f'Validation set: {len(X_val):>5} samples
({len(X_val)/len(X):.0%})')
print(f'Test set: {len(X_test):>5} samples
({len(X_test)/len(X):.0%})')
```

*# For classification with imbalanced classes, add stratify=y
to keep class ratios consistent across all three splits.*

Student

Why do we **split** the dataset into three parts?

ChatGPT • AI Assistant

Because we need to answer three different questions with three different pools of data:

1. **Training set** - data the model *learns from*.
2. **Validation set** - data used to *tune hyperparameters* and choose the best model.
3. **Test set** - untouched data used for a *final, honest evaluation* - one shot only.

Mixing these breaks the honesty of your results. Every professional ML project separates all three.

CONCEPT CARD

Typical split ratios:

1. **80 / 10 / 10** - most common for small-to-medium datasets.
2. **70 / 15 / 15** - slightly more validation and test data.
3. **60 / 20 / 20** - when you can afford larger validation and test sets.
4. **98 / 1 / 1** - used for very large datasets (100M+ samples); 1% is still huge.

Student

Why can't I just use the training data to evaluate?

ChatGPT • AI Assistant

EXAMPLE CARD

Because it lies to you:

1. A model that **memorizes** the training set gets 100% accuracy on it.
2. That same model may score 40% on new data - the true real-world performance.
3. Evaluating on training data hides this failure (called **overfitting**, Chapter 4).
4. The **test set** simulates "data the model will never see during development."
5. Only measuring on test data tells you how the model will perform in production.

🔗 ANALOGY

Train/val/test is like **studying for an exam**:

1. **Training set** = your textbook - read it, study it, learn from it.
2. **Validation set** = practice quizzes - check what you know, adjust your study plan.
3. **Test set** = the final exam - one shot, cannot look at it during study, tells the truth about your knowledge.

Studying from the exam paper is cheating. Testing on training data is the same crime.

⚠️ WARNING

NEVER touch the test set until the very end. Every peek at it during development leaks information and inflates your confidence. If you're evaluating on the test set repeatedly, you've turned it into a second validation set - you no longer have a true test.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Splitting AFTER preprocessing - you accidentally leak test-set statistics (mean, std) into your training pipeline.

Correct Way: Split FIRST, then compute preprocessing statistics on the training set only, and apply them to validation and test sets.

EXAMPLE CARD

What each set enables:

1. **Training** - the algorithm sees these labels and updates its parameters.
2. **Validation** - you compare 5 different models (e.g., 5 hyperparameter settings) and pick the best.
3. **Test** - after model selection, one final evaluation to report to stakeholders.

EXAM TIP CARD

Interview classic: *"Why do we need a separate validation set if we already have training and test?"* Winning answer: *"Because we need one dataset for learning*

parameters (training), one for tuning hyperparameters and model selection (validation), and one for a final unbiased evaluation (test). Reusing test for validation contaminates it."

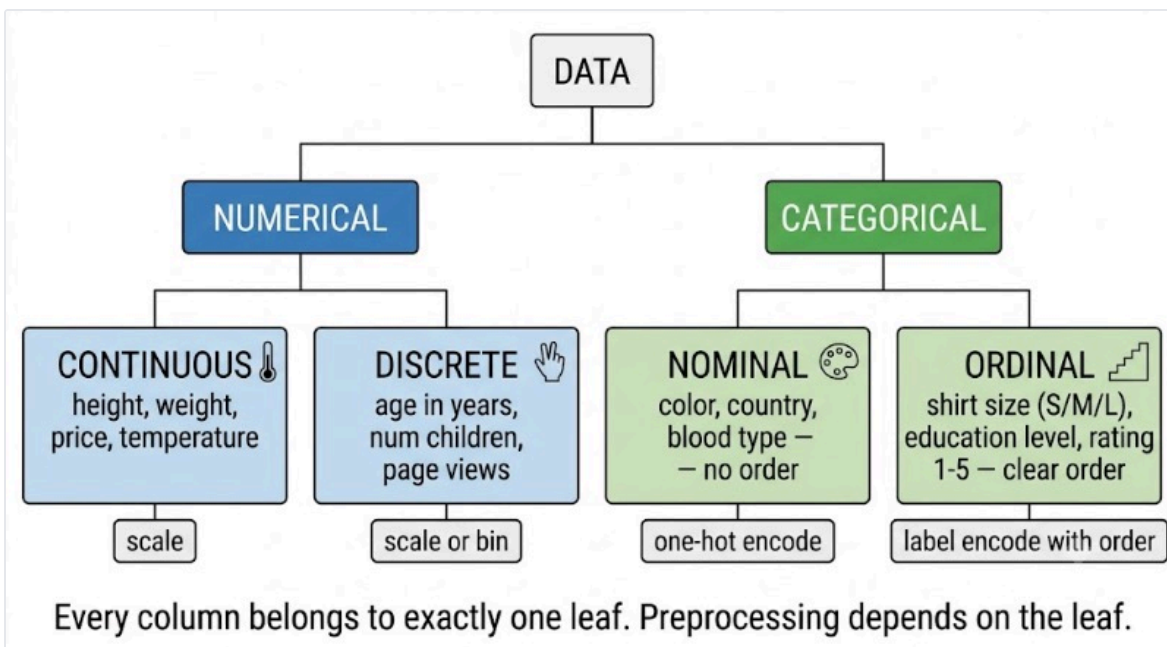
🔥 EXERCISE

Take any dataset. Use `train_test_split` twice to create 70/15/15 splits. Print sizes to verify.

[AI Prompt] Paste this into any chatbot: "Show me a Python snippet that splits a pandas DataFrame into 70% train, 15% validation, and 15% test using `train_test_split` from `scikit-learn`. Explain why you need TWO calls to `train_test_split`."

NUMERICAL VS CATEGORICAL DATA

ChatGPT • AI Assistant



The Four Fundamental Data Types

Student

What are the **main types** of data I'll encounter in tabular ML?

ChatGPT • AI Assistant

Two big families with two subtypes each. Every column in every dataset falls into one of these four buckets, and each bucket needs different preprocessing.

CONCEPT CARD

The four fundamental data types:

1. **Numerical - Continuous** - real numbers along a range. Height, weight, temperature, price. Any value in an interval is possible.
2. **Numerical - Discrete** - countable integers. Number of children, page views, orders per day. No fractions.
3. **Categorical - Nominal** - unordered categories. Color (red / blue / green), country, blood type. No natural order.
4. **Categorical - Ordinal** - ordered categories with meaningful ranking. T-shirt size (S / M / L / XL), education level, star rating.

Student

Why does the difference matter?

ChatGPT • AI Assistant

EXAMPLE CARD

Because each type needs different treatment:

1. **Continuous** - usually scaled (Section 2-8); models compute distances and gradients over these directly.
2. **Discrete** - treated like continuous most of the time; occasionally binned for tree-based models.
3. **Nominal** - must be **one-hot encoded** (Section 2-9). Cannot treat 'red' as "less than" 'blue.'
4. **Ordinal** - can be **label encoded** with the correct order preserved. "S < M < L" is meaningful.

Mislabel a nominal as ordinal and your model learns spurious "orderings" from arbitrary category codes.

🔗 ANALOGY

Data types are like **different kinds of measurements in a kitchen**:

- Continuous = weighing flour on a digital scale (2.345 kg).
- Discrete = counting eggs (3 whole eggs).

- Nominal = choosing a dish name (biryani / pizza / pasta - no order).
- Ordinal = spice level (mild / medium / hot / extra-hot - clear ranking).

⚠ WARNING

Watch out for **numbers that are actually categories** - postal codes, phone numbers, product IDs. They look numeric but arithmetic on them is meaningless. Treat them as nominal categorical.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Treating `zip_code` as a numerical feature. Model computes averages and gradients over ZIP codes as if $90210 > 10001$ by numeric value.

Correct Way: Treat ZIP codes as nominal categories. One-hot encode or use target encoding.

EXAMPLE CARD

A quick classification exercise for a typical dataset row:

1. `age = 27` → **continuous** (any real age possible).
2. `num_orders = 3` → **discrete** (whole count).
3. `city = 'Hyderabad'` → **nominal** (no order between cities).
4. `satisfaction = 'Very Satisfied'` → **ordinal** (Very Dissatisfied < Neutral < Very Satisfied).
5. `is_premium = True` → **binary nominal** (special case of nominal).

EXAM TIP CARD

Interview classic: *"When would you use one-hot encoding vs label encoding?"*

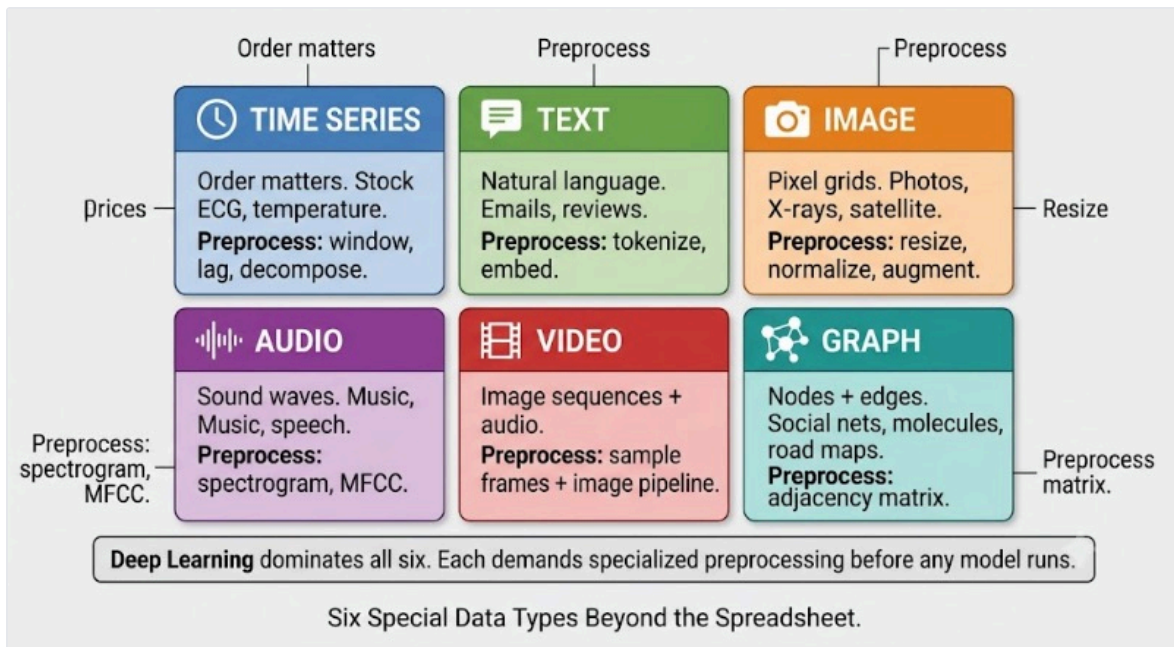
Winning answer: *"One-hot for nominal (unordered) categories to avoid spurious ordering; label encoding for ordinal categories where the order is meaningful and should be preserved."*

🔥 EXERCISE

Take any dataset (e.g., Titanic, Iris, house prices). For every column, classify it as continuous, discrete, nominal, or ordinal. Justify each.

[AI Prompt] Paste this into any chatbot: *"For each column in the Titanic dataset (PassengerId, Pclass, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked), classify it as continuous, discrete, nominal, or ordinal. Justify each choice."*

SPECIAL DATA TYPES — TIME SERIES, TEXT, IMAGE, AUDIO, VIDEO, GRAPH



Six Special Data Types Beyond the Spreadsheet

python – Loading three common special data types – time series, text, image

```
# =====
# TIME SERIES – never shuffle; split chronologically
# =====
import pandas as pd

ts_df = pd.read_csv('stock_prices.csv', parse_dates=['date'])
ts_df = ts_df.sort_values('date')

# Chronological split – 80% oldest for train, 20% newest for test
split_index = int(len(ts_df) * 0.8)
train_ts = ts_df.iloc[:split_index]
test_ts = ts_df.iloc[split_index:]

# =====
# TEXT – tokenize into words for classical ML
# =====
from sklearn.feature_extraction.text import CountVectorizer

reviews = [
    'Great product, loved it',
    'Terrible quality, disappointed',
    'Good value for money'
]

vectorizer = CountVectorizer()
```

```

X_text = vectorizer.fit_transform(reviews)
print(f'Vocabulary size: {len(vectorizer.vocabulary_)}')
print(f'Feature matrix shape: {X_text.shape}') # (samples, vocab_size)

# =====
# IMAGE - load, resize, normalize pixel values
# =====

from PIL import Image
import numpy as np

img = Image.open('cat.jpg')
img = img.resize((224, 224)) # standard input for many
CNNs
img_array = np.array(img) / 255.0 # scale pixels to [0, 1]
print(f'Image tensor shape: {img_array.shape}') # (224, 224, 3)

```

Student

What about **non-tabular data** - text, images, audio, video?

ChatGPT • AI Assistant

These are the **special data types** - the reason deep learning exists. They don't fit the tidy row-and-column shape and need specialized preprocessing before any model can use them.

CONCEPT CARD

Six special data types every ML engineer meets:

1. **Time series** - measurements over time (stock prices, temperature, ECG). Order matters. Cannot shuffle rows.
2. **Text** - natural language (emails, reviews, chat messages). Converted to numbers via tokenization + embeddings.
3. **Image** - pixel grids (photos, X-rays, satellite images). Represented as arrays of shape (height, width, channels).
4. **Audio** - waveforms sampled over time (music, speech). Often converted to spectrograms first.
5. **Video** - sequences of image frames + audio. Combines image and time-series challenges.

6. **Graph** - nodes and edges (social networks, molecules, road maps). Requires graph neural networks.

Student

Do any traditional ML algorithms work on these?

ChatGPT • AI Assistant

EXAMPLE CARD

Sometimes - with clever feature engineering. But deep learning DOMINATES special data types:

1. **Time series** - traditional ARIMA still works; LSTMs and Transformers dominate for complex patterns.
2. **Text** - bag-of-words was traditional; today's Transformers (BERT, GPT) crush it.
3. **Image** - hand-crafted features (edges, SIFT) were traditional; today's CNNs beat them by 30-50%.
4. **Audio** - MFCC features + traditional models were the norm; today's wav2vec and audio Transformers dominate.
5. **Video** - 3D-CNNs and Vision Transformers for action recognition, generation.
6. **Graph** - graph neural networks (GNNs) for drug discovery, fraud rings, recommendation.

🔗 ANALOGY

Special data types are like **different musical instruments**. A piano (tabular data) is versatile but needs a specific playing style.

A violin (time series), guitar (audio), drums (video), harp (graph) - each demands its own technique. Deep learning is the modern conservatory that trains models on all of them.

⚠️ WARNING

Time-series data has a **critical extra rule**: never shuffle before splitting. Chronological order encodes causality. Use `TimeSeriesSplit` in scikit-learn, not `train_test_split`.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Randomly shuffling time-series data before splitting. Your model learns to predict the past from the future - useless in production.

Correct Way: Split chronologically: train on older data, validate on middle period, test on latest data. This mimics real deployment.

EXAMPLE CARD

Quick preprocessing recipes:

1. **Text** - tokenize → build vocabulary → convert to token IDs → optionally embed.
2. **Image** - resize to fixed dimensions → normalize pixel values → augment (rotate, crop).
3. **Audio** - sample at consistent rate → compute spectrogram or MFCC → normalize.
4. **Time series** - check stationarity → decompose trend and seasonality → window into sequences.
5. **Video** - sample frames at a fixed rate → apply image preprocessing to each frame.
6. **Graph** - build adjacency matrix → compute node features → optionally add edge weights.

EXAM TIP CARD

Interview classic: *"How would you handle time series data differently from tabular data?"* Winning answer: *"Preserve chronological order - never shuffle. Use TimeSeriesSplit for validation."*

Check for stationarity, seasonality, and trends. Use rolling window features, lag features, and time-aware models like ARIMA, LSTM, or Transformers."

🔥 EXERCISE

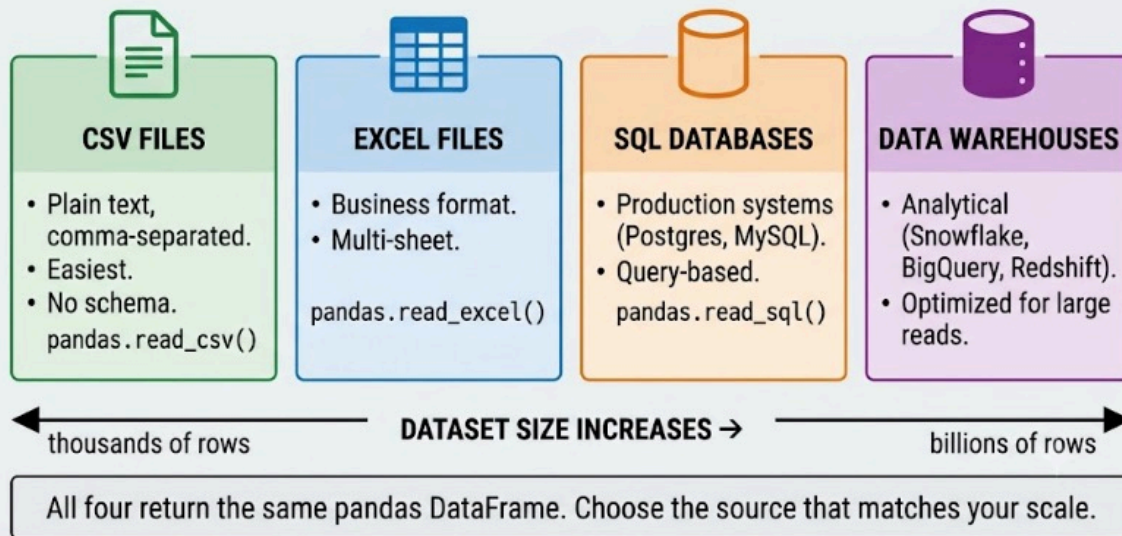
Pick one special data type (text, image, or audio). Download a small public dataset (movie reviews, MNIST digits, spoken digits). Load it in Python, print its shape, and describe how you'd preprocess it for ML.

[AI Prompt] Paste this into any chatbot: *"For each of the 6 special data types (time series, text, image, audio, video, graph), give me one real-world ML application, the standard preprocessing pipeline, and the dominant modern architecture used."*

DATA COLLECTION — FILES, DATABASES, WAREHOUSES

ChatGPT • AI Assistant

Structured Data Sources — From File to Warehouse



Structured Data Sources — From File to Warehouse

python – Load data from four structured sources – same pandas API

```
import pandas as pd
from sqlalchemy import create_engine

# =====
# 1) CSV file (the universal starter format)
# =====
df_csv = pd.read_csv('sales.csv')

# For huge files, iterate in chunks instead of loading all at once
chunks = pd.read_csv('big_sales.csv', chunksize=100_000)
small_pieces = [chunk[chunk['amount'] > 1000] for chunk in chunks]
df_big = pd.concat(small_pieces, ignore_index=True)

# =====
# 2) Excel file (business format, multiple sheets)
# =====
df_excel = pd.read_excel('quarterly_report.xlsx', sheet_name='Q3')

# Read ALL sheets into a dict of DataFrames
sheets_dict = pd.read_excel('report.xlsx', sheet_name=None)
for name, sheet in sheets_dict.items():
    print(f'{name}: {sheet.shape}')

# =====
# 3) SQL database (production system)
```

```
# =====
engine = create_engine('postgresql://user:pw@host:5432/mydb')
df_sql = pd.read_sql(
    'SELECT customer_id, revenue FROM orders WHERE year = 2026',
    engine
)

# =====
# 4) Data warehouse (large-scale analytics)
# =====
# Snowflake / BigQuery / Redshift look the same to pandas
# via SQLAlchemy – just a different connection string.
engine_snow = create_engine('snowflake://user:pw@account/db/schema')
df_wh = pd.read_sql('SELECT * FROM analytics.fact_sales', engine_snow)

# → All four return the same DataFrame API for downstream ML.
```

Student

Where does **real-world data** actually come from?

ChatGPT • AI Assistant

From four main categories of source. This subsection covers the **structured, static sources** - files, databases, and warehouses. The next covers **live sources** - APIs, sensors, and web scraping.

CONCEPT CARD

Structured static sources:

1. **CSV files** - the universal exchange format. Plain text, comma-separated. Easiest to load; no schema enforcement.
2. **Excel files** - spreadsheets with multiple sheets, formulas, styles. Common in business but messier than CSV.
3. **SQL databases** - production systems like PostgreSQL, MySQL, SQL Server. Structured, schema-enforced, queryable.
4. **Data warehouses** - analytical databases like Snowflake, BigQuery, Redshift. Optimized for large-scale reads.

Student

Which one should I start with?

ChatGPT • AI Assistant

EXAMPLE CARD

Rule of thumb by project stage:

1. **Learning / prototyping** → CSV files. Kaggle, UCI, and every tutorial uses them.
2. **Small business tools** → Excel files (accept the mess; convert to CSV early).
3. **Production apps** → SQL databases. Every real product has one.
4. **Analytics workloads at scale** → Data warehouses. Millions to billions of rows.
5. **A single project can pull from all four** - pandas `read_csv` , `read_excel` , `read_sql` all return DataFrames.

🔗 ANALOGY

Data sources are like **water sources**:

- CSV = bottled water (easy, portable, one-off).
- Excel = well water (nearby, workable, sometimes murky).
- SQL database = plumbed home tap (reliable, structured, always fresh).
- Data warehouse = municipal reservoir (huge scale, engineered for many consumers).

⚠️ WARNING

CSV has no type enforcement. A column of dates might be interpreted as strings; a numeric column with one bad row becomes text. Always verify types after loading with `df.dtypes` .

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Loading a huge CSV (2 GB+) into pandas without checking memory. Kills your Jupyter kernel.

Correct Way: Use `chunksize` to iterate over large CSVs, or switch to Parquet / Feather for large datasets.

EXAMPLE CARD

Common file formats you'll encounter:

1. **CSV** - plain text; widely supported but slow and untyped.
2. **JSON** - nested structure; common for API responses.

3. **Parquet** - columnar format; fast + compressed. Preferred for medium-large datasets.
4. **Excel (XLSX)** - business format; use `read_excel` sparingly.
5. **SQLite** - file-based SQL database; great for local experiments.

EXAM TIP CARD

Interview classic: *"How would you load a 100M-row dataset for ML?"* Winning answer: *"Prefer Parquet or a warehouse table over CSV. Use pandas with chunksize, or step up to Dask, Polars, or Spark for out-of-memory processing. Push heavy filtering and aggregation into SQL where possible."*

🔥 EXERCISE

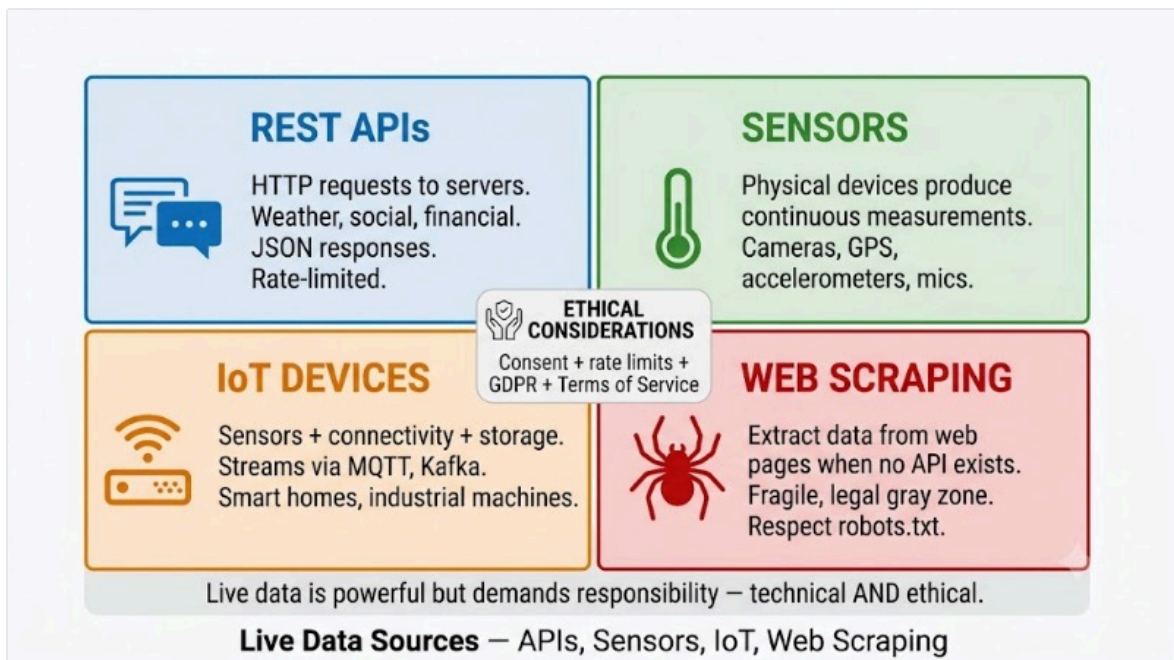
Download a CSV from Kaggle. Load it three ways: `pd.read_csv` , `pd.read_csv(chunksize=1000)` (iterative), and `pd.read_parquet` after converting. Time each. Notice the trade-off.

[AI Prompt] Paste this into any chatbot: *"Show me Python code to load data from 4 different sources into pandas DataFrames: a CSV file, an Excel file with multiple sheets, a PostgreSQL database via SQLAlchemy, and a BigQuery table. Explain the arguments for each."*

DATA COLLECTION — APIS, SENSORS, IOT, WEB SCRAPING



ChatGPT • AI Assistant



Live Data Sources — APIs, Sensors, IoT, Web Scraping

python – Fetching data from a REST API into a pandas DataFrame

```
import requests
import pandas as pd
import time

# =====
# Fetch current weather for 5 Indian cities
# =====
API_KEY = 'your_key_here'
CITIES = ['Hyderabad', 'Mumbai', 'Delhi', 'Chennai', 'Kolkata']

results = []
for city in CITIES:
    url = f'https://api.openweathermap.org/data/2.5/weather'
    params = {'q': city, 'appid': API_KEY, 'units': 'metric'}

    # Retry with exponential backoff on failure
    for attempt in range(3):
        try:
            response = requests.get(url, params=params, timeout=10)
            response.raise_for_status()
            data = response.json()
            results.append({
                'city': city,
                'temp_c': data['main']['temp'],
                'humidity': data['main']['humidity'],
```

```

        'weather': data['weather'][0]['main']
    })
    break # success - exit

retry loop
except requests.RequestException as e:
    wait = 2 ** attempt # 1, 2, 4
seconds
    print(f'Attempt {attempt+1} failed: {e}. Retry in
    {wait}s ... ')
    time.sleep(wait)

    time.sleep(1) # respect rate
limit

df = pd.DataFrame(results)
print(df)

```

Student

What about **live data** - from the internet or physical devices?

ChatGPT • AI Assistant

These are the **dynamic sources** - data that changes as you read it, often streamed in real time. Four main flavors: REST APIs, sensors, IoT devices, and web scraping. Each has its own ethical and technical considerations.

CONCEPT CARD

The four dynamic sources:

1. **REST APIs** - request data from a server over HTTP. Weather APIs, Twitter/X, GitHub, financial data. Usually JSON responses; often rate-limited.
2. **Sensors** - physical devices producing continuous measurements. Cameras, microphones, GPS, thermometers, accelerometers.
3. **IoT devices** - sensors + connectivity + storage/processing. Smart thermostats, industrial machines, wearables. Data streams via MQTT, Kafka, or cloud APIs.
4. **Web scraping** - extract data from web pages when no API exists. Fragile (page structure changes) and often legally gray.

Student

What are the ethical and legal considerations?

ChatGPT • AI Assistant

EXAMPLE CARD

Rules of the road:

1. **APIs** - respect rate limits; use official keys; read the terms of service.
2. **Sensors and IoT** - user consent for personal data (accelerometers, health monitors, location).
3. **Web scraping** - check `robots.txt` ; avoid personal data; don't hammer servers; some sites explicitly forbid scraping.
4. **All data** - comply with GDPR, DPDP Act 2023, and other data-protection regulations.
5. **When in doubt** - ask permission or use a public dataset instead.

🔗 ANALOGY

Live data sources are like **kinds of live journalism**:

- APIs = talking to an official press office (structured, sanctioned).
- Sensors = installing your own listening device (need consent).
- IoT = a network of embedded reporters (auto-streaming events).
- Web scraping = reading someone else's newspaper without permission (allowed sometimes, forbidden others).

⚠️ WARNING

Web scraping is a legal minefield in 2026 - the LinkedIn vs hiQ ruling, Reddit's API changes, and the news publishers' lawsuits against AI companies all revolve around it. Consult terms of service before scraping any commercial site.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Hitting a public API 1000 times a second without an API key. You get banned; the API team gets paged.

Correct Way: Read the docs. Use the API key. Respect the rate limit. Cache responses. Retry with exponential backoff on failures.

EXAMPLE CARD

Common Python libraries:

1. **APIs** → `requests` (simple), `httpx` (async), `aiohttp` (async).

2. **Sensors on Raspberry Pi / Arduino** → `GPIO` , vendor-specific SDKs.
3. **IoT streams** → `paho-mqtt` , `kafka-python` .
4. **Web scraping** → `BeautifulSoup` (parse HTML), `Selenium` (dynamic pages), `Scrapy` (large crawls).
5. **Storing streams** → append to a time-series DB, or accumulate into a CSV / Parquet.

EXAM TIP CARD

Interview classic: *"How would you collect data for a real-time fraud-detection model?"* Winning answer: *"Consume transaction events from a message broker (Kafka), enrich with customer features via a REST API, buffer in a streaming store (Kinesis or Kafka), and periodically batch to a data lake for training data."*

🔥 EXERCISE

Use `requests` to hit a free public API (weather, jokes, currency exchange). Parse the JSON, extract 2 fields, and put them in a pandas DataFrame. That's your first live data pipeline.

[AI Prompt] Paste this into any chatbot: *"Show me Python code that fetches weather data from the OpenWeatherMap API, converts the JSON response into a pandas DataFrame, and handles rate limits with exponential backoff. Include error handling."*

DATA CLEANING — MISSING VALUES, DUPLICATES, OUTLIERS

ChatGPT • AI Assistant

The Three Big Data Quality Issues – Visual View

MISSING VALUES

name	age	income
John Doe	NaN	NaN
John Doe	34	NaN
John Doe	NaN	NaN
NaN	NaN	NaN
John Doe	34	NaN

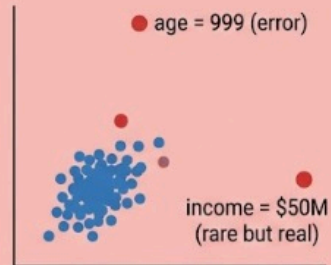
Options: drop row, drop column, impute with mean/median/mode, model-based fill.

DUPLICATE ROWS

name	age	colun	income
John Doe	34	Hyderabad	\$50k
John Doe	34	Hyderabad	\$50k
John Doe	34	Hyderabad	\$50k
John Doe	34	Hyderabad	\$50k
John Doe	34	Hyderabad	\$50k

Detect with `df.duplicated()`. Usually safe to drop.

OUTLIERS



Detect with IQR, z-score, or domain rules. Investigate BEFORE deleting.

Every real dataset has all three. Clean carefully – don't shortcut.

The Three Big Data Quality Issues – Visual View

python – Cleaning a dataset – missing values, duplicates, outliers

```
import pandas as pd
import numpy as np

# =====
# STEP 1: Explore the damage
# =====
print(df.isnull().sum())          # missing values per column
print(f'Duplicate rows: {df.duplicated().sum()}')
print(df.describe())              # numerical distributions

# =====
# STEP 2: Handle missing values
# =====
# Numerical: fill with median (robust to outliers)
for col in ['age', 'income', 'sqft']:
    df[col] = df[col].fillna(df[col].median())

# Categorical: fill with mode (most common value)
for col in ['city', 'category']:
    df[col] = df[col].fillna(df[col].mode()[0])

# Drop rows still missing critical fields
df = df.dropna(subset=['sale_price'])  # keep only rows with
label
```

```

# =====
# STEP 3: Remove duplicates
# =====
before = len(df)
df = df.drop_duplicates()
print(f'Removed {before - len(df)} duplicate rows')

# =====
# STEP 4: Cap outliers using IQR method
# =====
def cap_outliers(series):
    Q1, Q3 = series.quantile([0.25, 0.75])
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    return series.clip(lower=lower, upper=upper)

for col in ['income', 'sale_price']:
    df[col] = cap_outliers(df[col])

print(f'Cleaned dataset shape: {df.shape}')

```

Student

Now the hard part - **cleaning** dirty data. What are the three big issues?

ChatGPT • AI Assistant

Every real dataset has three problems: **missing values**, **duplicate rows**, and **outliers**. Handling each correctly is more art than science, but there are professional patterns for all three.

CONCEPT CARD

The three big cleaning issues:

1. **Missing values** - cells with **NaN**, **null**, or empty strings. Options: drop, impute (fill with mean / median / mode), or model-based imputation.
2. **Duplicate rows** - identical or near-identical rows. Usually accidental (double-insert, join bug). Almost always remove.
3. **Outliers** - data points far from the typical distribution. Sometimes errors (typo in age = 999), sometimes real (a legitimate whale customer). Never blindly delete.

How do I decide what to do with missing values?

ChatGPT • AI Assistant

EXAMPLE CARD

The four common strategies:

1. **Drop rows** - safe when missingness is rare (<5% of rows). Simplest but loses data.
2. **Drop columns** - if a column has too many missing values (>50%), consider removing it entirely.
3. **Impute with statistic** - fill numerical NaNs with mean or median; fill categorical NaNs with mode. Fast and effective.
4. **Model-based imputation** - use a mini ML model to predict missing values based on other columns. Best but slowest.

Student

How do I detect outliers?

ChatGPT • AI Assistant

EXAMPLE CARD

The three standard methods:

1. **IQR method** - anything below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is an outlier. Classical and robust.
2. **Z-score method** - anything more than 3 standard deviations from the mean. Assumes normal distribution.
3. **Domain rules** - age > 120, price < 0, height > 3 meters. Business logic catches errors that statistics miss.

🔗 ANALOGY

Data cleaning is like **cooking with fresh vegetables**:

1. Missing values = a wilted head of lettuce - decide to compost (drop) or salvage (impute).
2. Duplicates = two heads of the same lettuce accidentally in your basket - remove one.

3. Outliers = one giant pumpkin among small tomatoes - could be a mistake or a legitimate special ingredient. Investigate before discarding.

⚠ WARNING

Never impute using statistics from the whole dataset before splitting. That leaks test-set information into training. Compute mean/median on the training set only, then apply to validation and test.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: `df.fillna(df.mean())` on the full dataset before train/test split. Test set info leaks into training.

Correct Way: Split first. Compute mean on train only. Apply that same mean to fill NaNs in train, val, and test.

EXAMPLE CARD

What to do when outliers appear:

1. **Investigate first** - was it a data entry error (999 = missing), a unit-mixup (kilograms vs grams), or a legitimate rare event?
2. **Domain rules first** - apply obvious constraints (age > 120 is impossible).
3. **Cap / winsorize** - replace outliers with the 1st or 99th percentile value.
4. **Transform** - apply log or square-root transform to compress the tail.
5. **Drop as last resort** - only when confident the point is an error, not a rare-but-real event.

EXAM TIP CARD

Interview classic: *"How would you handle missing values in a dataset?"* Winning answer: *"Depends on the amount and mechanism. Drop rows for rare missingness; drop columns above 50% missing; impute with median (numerical) or mode (categorical) for moderate cases; use model-based imputation when patterns are complex."*

Always compute imputation values on training data only to avoid leakage."

🔥 EXERCISE

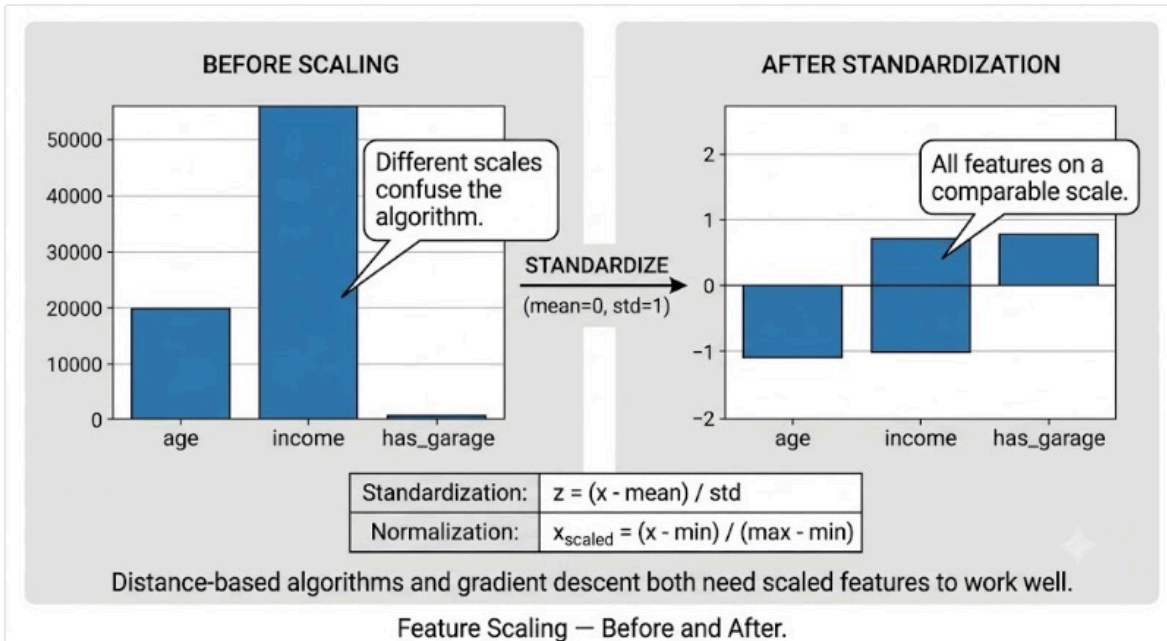
Take any dataset. Report: (a) percentage of missing values per column, (b) number of duplicate rows, (c) outliers in each numeric column using IQR method. Write one sentence about how you'd handle each issue.

[AI Prompt] Paste this into any chatbot: *"Show me a Python function `clean_dataframe(df)` that removes duplicates, handles missing values with median*

imputation for numeric and mode for categorical, and caps outliers at the 1st and 99th percentiles. Include comments explaining each step."

FEATURE SCALING — STANDARDIZATION AND NORMALIZATION

ChatGPT • AI Assistant



Feature Scaling — Before and After

python – Scaling features correctly with train/test separation

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split

# =====
# Split BEFORE scaling to prevent data leakage
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# STANDARDIZATION – most common choice
# =====
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # FIT on train only
```

```

X_test_scaled = scaler.transform(X_test)          # transform test with
train stats

print('Train mean per feature:',
X_train_scaled.mean(axis=0).round(3))
print('Train std per feature:', X_train_scaled.std(axis=0).round(3))

# =====
# NORMALIZATION – for bounded input requirements
# =====
minmax = MinMaxScaler(feature_range=(0, 1))
X_train_norm = minmax.fit_transform(X_train)
X_test_norm = minmax.transform(X_test)

print('Train min per feature:', X_train_norm.min(axis=0))
print('Train max per feature:', X_train_norm.max(axis=0))

# =====
# For deployment: save the scaler to reuse on new data
# =====
import joblib
joblib.dump(scaler, 'scaler.pkl')    # apply this in production too

```

Student

Why do I need to **scale** features?

ChatGPT • AI Assistant

Because many ML algorithms assume features are on similar scales. A feature ranging from 0 to 1 (like `has_garage`) and another ranging from 0 to 1,000,000 (like `sale_price`) confuse distance-based algorithms and slow down gradient descent.

Feature scaling puts every feature into a comparable range.

CONCEPT CARD

The two dominant scaling techniques:

1. **Standardization (Z-score scaling)** - transform to mean 0, standard deviation 1.
Formula: $(x - \text{mean}) / \text{std}$. Works well when data is roughly normal; robust to some outliers.

2. **Normalization (Min-Max scaling)** - squash to a fixed range, typically [0, 1].

Formula: $(x - \min) / (\max - \min)$. Sensitive to outliers; use when you need bounded features.

Student

Which one should I use?

ChatGPT • AI Assistant

EXAMPLE CARD

Quick decision guide:

1. **Standardization** → default for most ML algorithms (linear regression, logistic regression, SVM, k-means, PCA, neural nets).
2. **Normalization** → use when the algorithm requires bounded inputs (some neural nets, image pixel scaling to [0, 1]).
3. **Neither** → tree-based models (decision trees, random forests, gradient boosting) are scale-invariant. No scaling needed.
4. **Robust scaling** → when outliers dominate; uses median and IQR instead of mean and std.

Student

Does scaling matter that much?

ChatGPT • AI Assistant

⚠ WARNING

Yes - dramatically:

1. **Distance-based algorithms** (k-NN, k-means, SVM) can produce completely wrong results without scaling.
2. **Gradient descent** converges 10-100× faster with scaled features.
3. **Regularization** (L1, L2) penalizes large coefficients - unfair on features with large scales.
4. **Neural networks** train unstably without scaled inputs.

💡 ANALOGY

Feature scaling is like **converting to a common currency** before adding bills. You can't meaningfully add ₹100 + \$10 + ¥1000. Convert them all to one currency first. Same for features - scale them into one comparable range.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Fitting the scaler on the FULL dataset before splitting. Test-set statistics leak into training.

Correct Way: `scaler.fit(X_train)` - fit ONLY on training data. Then `scaler.transform(X_val)` and `scaler.transform(X_test)` using the training statistics.

EXAMPLE CARD

Which algorithms need scaling:

1. **Need scaling:** Linear/logistic regression, SVM, k-NN, k-means, PCA, neural networks.
2. **Scale-invariant:** Decision trees, random forests, gradient boosting (XGBoost, LightGBM, CatBoost).
3. **When in doubt:** Scale. Cost is minimal; benefit can be huge.

EXAM TIP CARD

Interview classic: *"What's the difference between standardization and normalization?"* Winning answer: *"Standardization rescales to mean 0, standard deviation 1 - good for algorithms assuming normal-like distributions. Normalization rescales to a fixed range like [0, 1] - good when you need bounded inputs.*

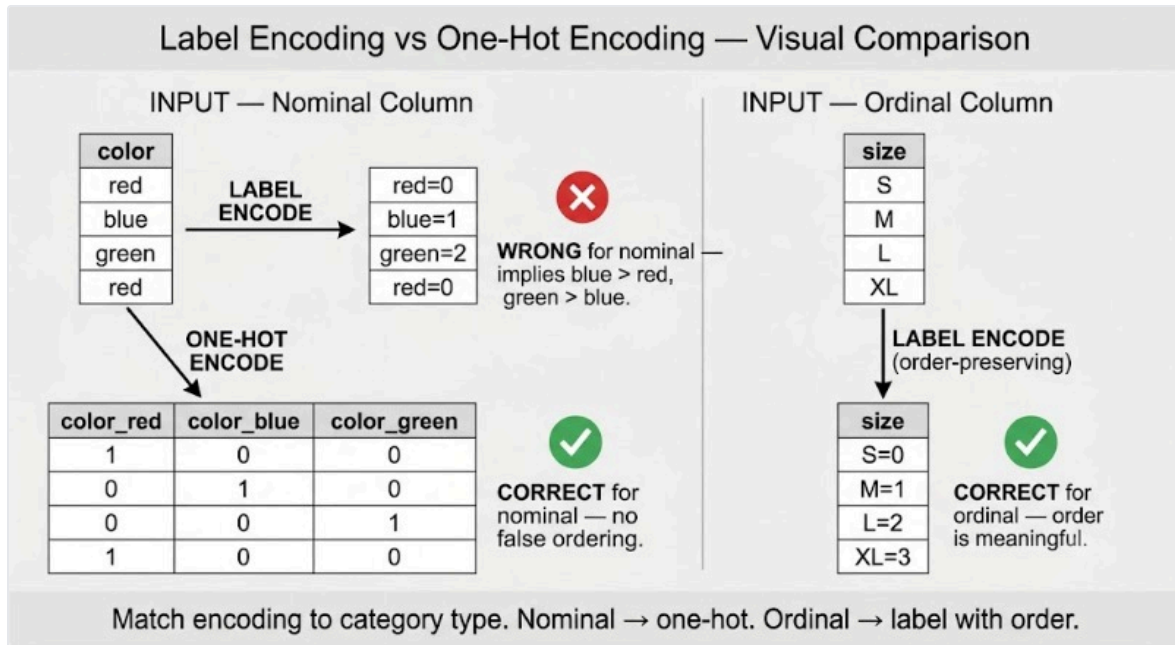
Standardization is the safer default."

🔥 EXERCISE

Load any dataset with numeric features. Apply both `StandardScaler` and `MinMaxScaler`. Print the min, max, mean, and std of each feature before and after. Notice the difference.

[AI Prompt] Paste this into any chatbot: *"Show me Python code that scales training and test data correctly using StandardScaler from sklearn - including how to avoid data leakage. Then explain why fitting the scaler on the full dataset before splitting is wrong."*

ENCODING AND FEATURE ENGINEERING



Label Encoding vs One-Hot Encoding — Visual Comparison

python – Encoding categorical features and engineering new ones

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# =====
# LABEL ENCODING for ordinal columns (order matters)
# =====
size_order = {'S': 0, 'M': 1, 'L': 2, 'XL': 3}
df['size_encoded'] = df['size'].map(size_order)

# =====
# ONE-HOT ENCODING for nominal columns (no order)
# =====
# Pandas quick way
df_encoded = pd.get_dummies(
    df,
    columns=['color', 'city'],
    prefix=['color', 'city'],
    drop_first=True
) # avoid dummy-variable trap

# Sklearn way (works inside pipelines)
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, drop='first')
ohe.fit(df[['color']])
```

```

encoded = ohe.transform(df[['color']])

# =====
# FEATURE ENGINEERING – creating new signals from raw data
# =====

df['order_date'] = pd.to_datetime(df['order_date'])

# Temporal features
df['hour'] = df['order_date'].dt.hour
df['day_of_week'] = df['order_date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
df['month'] = df['order_date'].dt.month

# Interaction / derived features
df['unit_price'] = df['total_amount'] / df['quantity']
df['price_per_sqft'] = df['sale_price'] / df['sqft']
df['bedrooms_per_sqft'] = df['bedrooms'] / df['sqft']

# =====
# FEATURE SELECTION – keeping the useful ones
# =====

from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(score_func=f_regression, k=10)
X_selected = selector.fit_transform(X, y)
print(f'Selected features:
{X.columns[selector.get_support()].tolist()}')

```

Student

How do I turn **text categories** into numbers a model can use?

ChatGPT • AI Assistant

Two dominant techniques: **label encoding** for ordinal categories, **one-hot encoding** for nominal categories. Then we'll look at **feature engineering** and **feature selection** - the creative and pragmatic sides of ML.

CONCEPT CARD

The two encoding techniques:

1. **Label encoding** - replace each category with an integer. $S \rightarrow 0$, $M \rightarrow 1$, $L \rightarrow 2$, $XL \rightarrow 3$. Preserves order (great for ordinal), but implies order (bad for nominal).
2. **One-hot encoding** - create one binary column per category. $color=red$ becomes three columns: $color_red=1$, $color_blue=0$, $color_green=0$. No implied order; adds many columns.

Student

When do I use each?

ChatGPT • AI Assistant

EXAMPLE CARD

Simple rule:

1. **Ordinal categories** (S/M/L/XL, low/med/high) → **label encoding**. Order is meaningful.
2. **Nominal categories** (color, city, product type) → **one-hot encoding**. No implied order.
3. **High-cardinality nominals** (100+ categories) → one-hot creates too many columns. Alternatives: target encoding, hashing trick, embeddings.

Student

What's **feature engineering**?

ChatGPT • AI Assistant

Feature engineering = creating new features from existing ones to make patterns easier for the model. It's the #1 skill that separates a mediocre ML engineer from a great one.

EXAMPLE CARD

Real feature engineering patterns:

1. **From timestamp** → extract hour, dayofweek, month, isweekend, isholiday.
2. **From price + quantity** → compute $total_amount$, $unit_price$, $discount_pct$.
3. **From two coordinates** → compute distance, direction, iswithincity_limits.
4. **From full name** → extract title (Mr / Ms / Dr), first name, last name, length.
5. **From address** → extract ZIP, city, state, is_apartment.

6. **Interaction features** - $\text{bedrooms} \times \text{sqft}$, $\text{price} / \text{bedrooms}$ (price per bedroom).

Student

What about **feature selection**?

ChatGPT • AI Assistant

Feature selection = choosing which of many features to actually use. Fewer features often mean faster training, less overfitting, and better interpretability.

EXAMPLE CARD

Three common feature selection methods:

1. **Filter methods** - use statistical tests (correlation, chi-square, ANOVA) to score each feature.
2. **Wrapper methods** - try different feature subsets; keep the ones that improve model performance.
3. **Embedded methods** - use algorithms that select features internally (Lasso regression, tree-based feature importance).

🔗 ANALOGY

Feature engineering is like **preparing ingredients before cooking**. Raw carrots are hard to chew; chopped carrots are perfect.

The chef (algorithm) works much better with well-prepared ingredients (features). Selection is like **not putting every ingredient in the fridge into one dish** - some help, some hurt.

⚠️ WARNING

Don't create features from the label. If your feature depends on y , the model "predicts" trivially in training and fails in production. This is another form of leakage.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Applying one-hot encoding to a nominal column with 10,000 unique values (like `user_id` or `product_sku`). Creates a giant sparse matrix; models struggle.

Correct Way: For high-cardinality nominals, use target encoding (replace category with mean label value per category) or embeddings.

EXAM TIP CARD

Interview classic: *"How would you engineer features from a raw dataset?"* Winning answer: *"Look for temporal patterns (extract day/hour), interactions (products of features), aggregations (mean per group), text length, and domain-specific transforms. Use domain knowledge - a house's price per square foot is more informative than either alone."*

EXERCISE

Take any dataset with a timestamp column. Create 5 new features from it. Compute correlation of each with the label. Notice how engineered features often correlate stronger than the raw timestamp.

[AI Prompt] Paste this into any chatbot: *"For a Kaggle house-price prediction dataset, propose 15 engineered features (from raw columns like YearBuilt, GrLivArea, GarageCars, Neighborhood). For each, explain the intuition and how you'd compute it."*

CHAPTER REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

You now speak the language of ML data - from vocabulary to hands-on cleaning and preparation. This chapter is what data engineers actually *do* every day.

- **ML TERMINOLOGY:** Dataset = all examples. Sample / observation / row = one example. Feature / attribute / column = one input variable. Label / target = the output to predict. Dimension = number of features.
- **TRAINING, VALIDATION, AND TEST SETS:** Three splits with three purposes - training (learn parameters), validation (tune hyperparameters), test (final honest evaluation). Typical ratios 70/15/15 or 80/10/10. Never peek at test until the end.
- **NUMERICAL VS CATEGORICAL DATA:** Four types - continuous (real numbers), discrete (integers), nominal (unordered categories), ordinal (ranked categories). Each type needs different preprocessing.

- SPECIAL DATA TYPES: Time series (order matters - never shuffle), text (tokenize + embed), image (pixel arrays), audio (spectrograms), video (frames + audio), graph (nodes + edges). Deep learning dominates all six.
- DATA COLLECTION FROM STRUCTURED SOURCES: CSV, Excel, SQL databases, and data warehouses - same pandas API (`readcsv`, `readexcel`, `read_sql`). Choose based on scale.
- DATA COLLECTION FROM LIVE SOURCES: REST APIs, sensors, IoT streams, web scraping - richer but demand ethical care (rate limits, consent, robots.txt, GDPR).
- DATA CLEANING: Missing values (drop or impute), duplicates (usually remove), outliers (investigate before deleting). Compute imputation and scaling stats on training data only to prevent leakage.
- FEATURE SCALING: Standardization (mean 0, std 1) - the default. Normalization (0-1 range) - for bounded inputs. Tree-based models don't need scaling.
- ENCODING AND FEATURE ENGINEERING: Label encoding for ordinal, one-hot for nominal. Engineer features from timestamps, coordinates, interactions. Select features via filter, wrapper, or embedded methods.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You now hold the practical toolkit of an ML data engineer - the ability to source, clean, encode, and prepare any dataset for training. Real ML projects live and die at this stage.

In the next chapter - **Mathematics and Data Visualization for ML** - we shift from data preparation to the mathematical foundations. Statistics (mean, median, variance, correlation), probability (Bayes' theorem), linear algebra (vectors, matrices, eigenvalues), and calculus (derivatives, gradients, gradient descent) - plus the visualization tools (histograms, scatter plots, heatmaps) that reveal data patterns before you model.

Onward.